UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

IF 451
Advance Web Programming

Meet9 - Middleware & Routing in Express

Wirawan Istiono, S.Kom., M.Kom

Meet 9

Objective:

Sub-CLO0714 - Students are able to implement Node JS Framework to build Web application (C3, C6)

OUTLINE

- *Static files serving.*
- *Built-in & custom middleware*

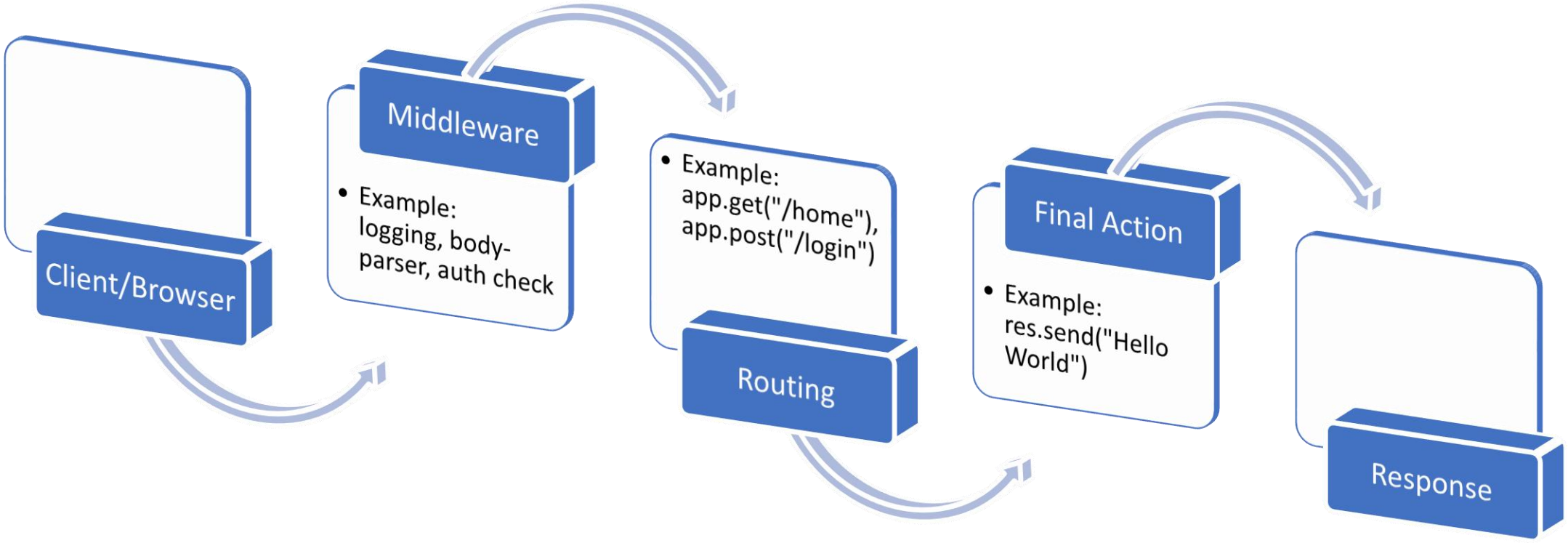
Introduction

What is MiddleWare?

Middleware functions are functions that sit between the request and the response cycle in ExpressJS. They have access to the req (request), res (response), and the next function (used to pass control to the next middleware).

- Middleware is a function that runs on every request before the route handler.
- Middleware can **modify** request and response objects.
- Middleware can perform tasks like:
  - Logging requests
  - Authentication and authorization
  - Parsing request bodies (e.g., express.json(), express.urlencoded())
  - Handling errors
- Middleware can be **application-level**, **router-level**, **built-in**, or **third-party**.

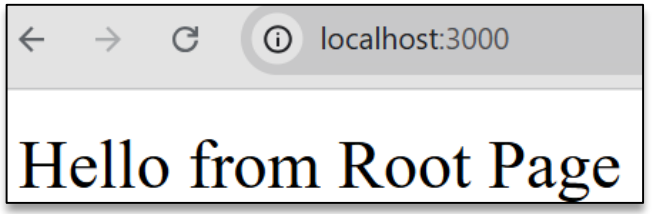
ExpressJS Request-Response Flow



Sample simple MiddleWare

```
const express = require("express");
const app = express();

app.use((req, res, next) => {
    console.log("From MiddleWare");
    console.log("URL: " + req.url);
    console.log("Method: " + req.method);
    next();
});

app.get("/", (req, res) => {
    res.send("Hello from Root Page");
});

app.get("/home", (req, res) => {
    res.send("This is Home Page");
});

app.listen(3000, () => console.log("http://localhost:3000"));
```

visit: localhost:3000




```
From MiddleWare
URL: /
Method: GET
```

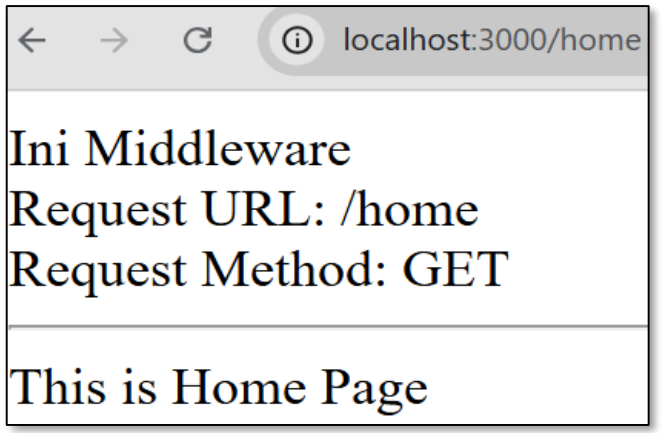
visit: localhost:3000/home


```
From MiddleWare
URL: /home
Method: GET
```

Sample simple MiddleWare to Website

```
const express = require("express");
const app = express();

app.use((req, res, next) => {
    let isiMidWare = "Ini Middleware<br>";
    isiMidWare += "Request URL: "+req.url+"<br>";
    isiMidWare += "Request Method: "+req.method+"<br>";
    isiMidWare += "<hr>";

    res.locals.info = isiMidWare;
    next();
});

app.get("/", (req, res) => {
  res.send(res.locals.info + "Hello from Root Page");
});
app.get("/home", (req, res) => {
  res.send(res.locals.info + "This is Home Page");
});
```

visit: localhost:3000/home




```
app.listen(3000, () =>
console.log("http://localhost:3000"
));
```

Static files serving

source: https://expressjs.com/en/starter/static-files.html

What are Static Files?

Static files are files that do not change dynamically on the server. They are usually **assets** like:

- HTML files
- CSS stylesheets
- JavaScript files (client-side)
- Images (PNG, JPG, GIF, SVG, etc.)
- Fonts (TTF, WOFF, etc.)

Express provides a built-in middleware called **express.static** to serve those files directly. This means, when a user requests a file (like **/style.css**), Express automatically looks inside a specific folder and sends the file to the browser — no need to write custom routes.

Sample express static (1)

Suppose you have this project structure:

```
project/
│
├── app.js
└── public/
    ├── index.html
    ├── home.html
    ├── style.css
    └── script.js
```

Below is sample script in **app.js**

```
const express = require("express");
const app = express();

app.use(express.static("public"));

app.listen(3000, () => {
  console.log("http://localhost:3000");
});
```

Sample express static (2)

Below is sample script in **index.html**

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <link rel="stylesheet" href="style.css">
    <title>Sample Index html page</title>
</head>
<body>
    <h1>Ini adalah body html</h1>
    <div id="txtKet">Akan diisi tulisan JS</div>
    <button id="btnKeHome" >Pindah ke home</button>
    <script src="script.js"></script>
</body>
</html>
```

Below is sample script in **home.html**

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <link rel="stylesheet" href="style.css">
    <title>Sample Home html page</title>
</head>
<body>
    <p>ini adalah halaman home</p>
</body>
</html>
```

Sample express static (3)

Below is sample script in **style.css**

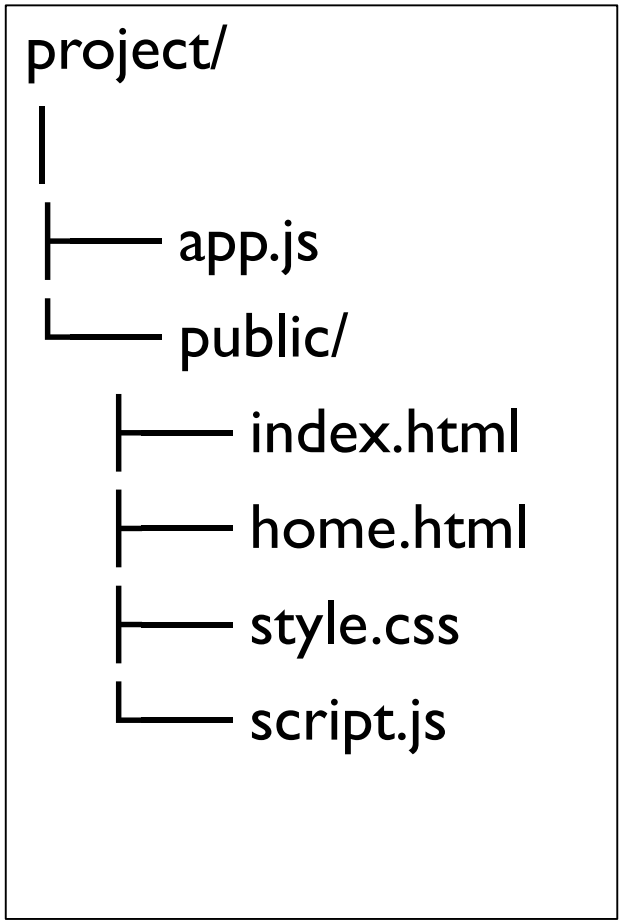
```
body {
    background-color:grey;
}
h1 {
    background-color:yellow;
    color: blue;
}
p{
    font-style: italic;
}
```

Below is sample script in **script.js**

Note: Sample code Javascript below is not javascript in express or nodeJS, because it's run in client side not server side

```
document.addEventListener("DOMContentLoaded", ()=>{
    document.getElementById("txtKet").innerHTML = 'Halo ini dari JS';

    document.getElementById("btnKeHome").addEventListener("click",()=> {
        alert("ini dari halaman js");
        window.location.href="home.html";
    });
});
```

Sample express static (4)



If you input:

http://localhost:3000, the page will open **index.html**

http://localhost:3000/home.html, the page will open **home.html**

http://localhost:3000/style.css, the page will open **style.css**

http://localhost:3000/script.js, the page will open **script.js**

Conclusion express.static

- express.static isn't dynamic routing like app.get("/profile", ...), but rather **provides static files (frontend assets)** so they can be accessed directly via URL.
- Examples of static files: index.html, style.css, script.js, images (.png, .jpg), fonts, etc.

With app.use(express.static("public")), Express will:

- Find files from the public/ folder.
- If a request is made to http://localhost:3000/index.html, the public/index.html file is sent directly to the browser.
- So we don't need to create manual routes for each file.

Files that should be placed in the public folder

All frontend files that must be accessed directly by the browser:

- index.html
- style.css
- script.js
- Images (.jpg, .png, .svg)
- Fonts (.ttf, .woff)

This file is intentionally opened to the public, because users need to access it so that the website can appear normally.

Typically the Express project structure is separated like this:

```
project/
├── app.js          → server backend
├── routes/         → routing logic
├── models/         → database models
├── public/         → frontend files (public access)
│     ├── index.html
│     ├── style.css
│     └── script.js
├── .env            → config (not public)
└── package.json
```

Built-in & custom middleware

What is Built-in Middleware?

Built-in Middleware is the middleware is **already available directly in Express**, so we just have to use it.

Sample:

1. express.static → (done)
2. express.json() → to accept requests with a JSON-formatted body.
3. express.urlencoded() → to accept requests with an HTML form body (x-www-form-urlencoded).

Sample express.urlencoded (1)

1. Create html file with name: **sample_form.html** in **public** folder
2. Write the sample code beside

```
project/
|
├── app.js
└── public/
    └── sample_form.html
```

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Sample Form URL Encoded</title>
</head>
<body>
    <form action="http://localhost:3000/form" method="POST">
        NIM: <input type="text" name="nim"><br>
        Nama: <input type="text" name="nama"><br>
        Jurusan: <input type="text" name="jurusan"><br>
        <button type="submit">Kirim</button>
    </form>
</body>
</html>
```

Sample express.urlencoded (2)

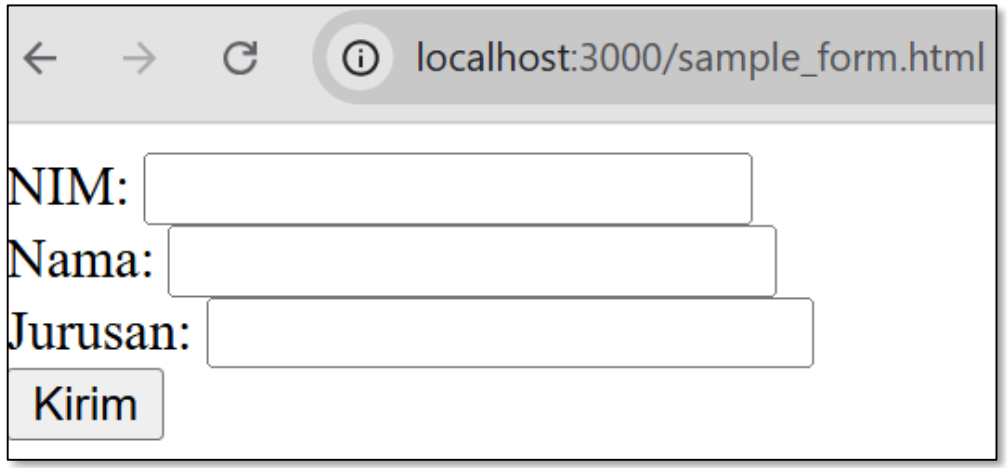
1. Create js file with name: **app.js** in **root** folder
2. Write the sample code beside

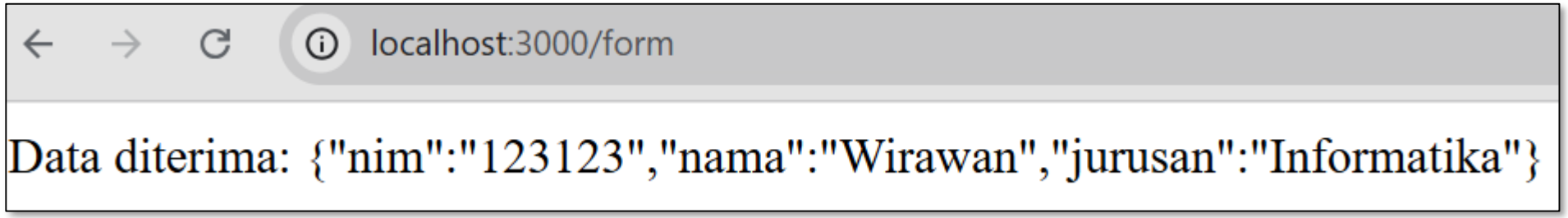
```
project/
│
├── app.js
└── public/
    └── sample_form.html
```

```
const express = require("express");
const app = express();

app.use(express.urlencoded({ extended: true }));
app.use(express.static("public"));

app.post("/form", (req, res) => {
    console.log(req.body);
    res.send("Data diterima: " + JSON.stringify(req.body));
});

app.listen(3000, () => {
    console.log("http://localhost:3000");
});
```

Sample express.urlencoded (3)

- Call the sample_form.html with http://localhost:3000/sample_form.html
- Fill the form and see the result





What is Custom Middleware

Custom middleware is used for parsing the body of a request (especially POST).

Middleware Characteristics in ExpressJS

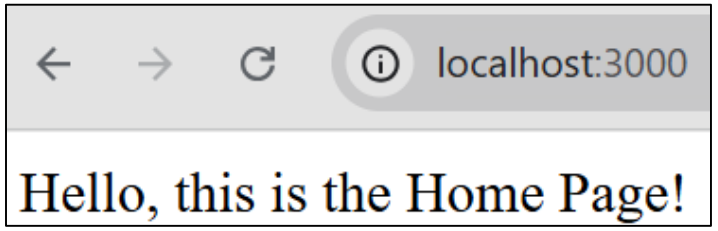
1. Has a 3-parameter signature: (req, res, next)
2. Must call next() so the request can be forwarded to the next middleware or route handler.

Custom Middleware means middleware that we create ourselves according to our needs, for example for logging, authentication, validation, etc.

Sample Custom Middleware: Logging

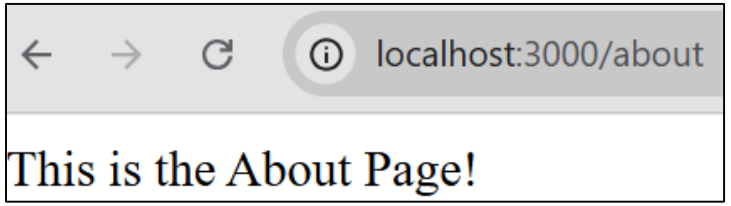
```
const express = require("express");
const app = express();

// Custom Middleware: Logging
const myLogger = (req, res, next) => {
  console.log(`Request Method: ${req.method}, URL:
${req.url}, Time: ${Date.now()}`);
  next();
};
app.use(myLogger);

app.get("/", (req, res) => {
  res.send("Hello, this is the Home Page!");
});
app.get("/about", (req, res) => {
  res.send("This is the About Page!");
});

app.listen(3000, () => {
  console.log("http://localhost:3000");
});
```

When you visit: http://localhost:3000




```
Request Method: GET, URL: /, Time: 1757579521500
```

When you visit: http://localhost:3000/about




```
Request Method: GET, URL: /about, Time: 1757579383984
```

Sample Custom Middleware:Validasi Form (1)

Create **app.js** new file, and write the sample code beside:

```
const express = require("express");
const app = express();

app.use(express.urlencoded({ extended: true }));

// Custom Middleware untuk validasi form
const validateForm = (req, res, next) => {
  const { nama, jurusan } = req.body;
  if (!nama || !jurusan) {
    return res.status(400).send("Nama dan Jurusan wajib diisi!");
  }
  next();
};

app.get("/", (req, res) => {
  res.sendFile(__dirname + "/index.html");
});
app.post("/form", validateForm, (req, res) => {
  const { nama, jurusan } = req.body;
  res.send("Data berhasil diterima!<br>Nama:"+nama+" Jurusan: "+jurusan);
});

app.listen(3000, () => {
  console.log("http://localhost:${port}");
});
```

Sample Custom Middleware:Validasi Form (2)

Create **sample_midvalidasi.html** new file, and write the sample code beside:

How to

1. Visit http://localhost:3000/
2. Fills out the form and submit.
3. The validateForm middleware checks whether the name and major fields are empty.
   - If empty → the response is 400 Bad Request.
   - If filled → the request continues to the /submit route handler.

```
<!DOCTYPE html>
<html>
<head>
  <title>Middleware Validasi Form</title>
</head>
<body>
  <form action="http://localhost:3000/form" method="POST">
    Nama: <input type="text" name="nama" /><br><br>
    Jurusan: <input type="text" name="jurusan" /><br><br>
    <button type="submit">Kirim</button>
  </form>
</body>
</html>
```

NEXT WEEK'S OUTLINE

- REST API with Express + MySQL Database

REFERENCES

- B. Lawson and J. Encinas, Node.js Web Development, 6th ed., Packt Publishing, 2023
- R. Raj, Learning Node.js Development, Packt Publishing, 2018.
- E. Cantelon, M. Harter, T. Holowaychuk, and N. Rajlich, Node.js in Action, 2nd ed., Manning Publications, 2017.
- Maulana, a., bau, r.t.r., munawar, z., setiawan, r., aisa, s., istiono, w., irfan, a., pratama, y.a., ramadhany, e.d., pomalingo, s. And permana, a.a., 2023. *Pemrograman web 101: memahami dasar-dasar untuk mengembangkan situs web*. Get press indonesia.
- The Node.js Foundation, "Node.js Documentation," [Online]. Available: https://nodejs.org/en/docs/. [Accessed: Jun. 5, 2025].
- Express.js Team, "Express.js Documentation," [Online]. Available: https://expressjs.com/. [Accessed: Jun. 5, 2025].

Visi

Menjadi Program Studi Strata Satu Informatika **unggulan** yang menghasilkan lulusan **berwawasan internasional** yang **kompeten** di bidang Ilmu Komputer (*Computer Science*), **berjiwa wirausaha** dan **berbudi pekerti luhur**.

INFORMATIKA
UMN
UNIVERSITAS MULTIMEDIA NUSANTARA

Misi

1. Menyelenggarakan pembelajaran dengan teknologi dan kurikulum terbaik serta didukung tenaga pengajar profesional.
2. Melaksanakan kegiatan penelitian di bidang Informatika untuk memajukan ilmu dan teknologi Informatika.
3. Melaksanakan kegiatan pengabdian kepada masyarakat berbasis ilmu dan teknologi Informatika dalam rangka mengamalkan ilmu dan teknologi Informatika.